

UNIT 1

Software:

Basically, software is a set of programs or collection of computer programming which performed set of task and software will be developed by developer team based on customer requirement.

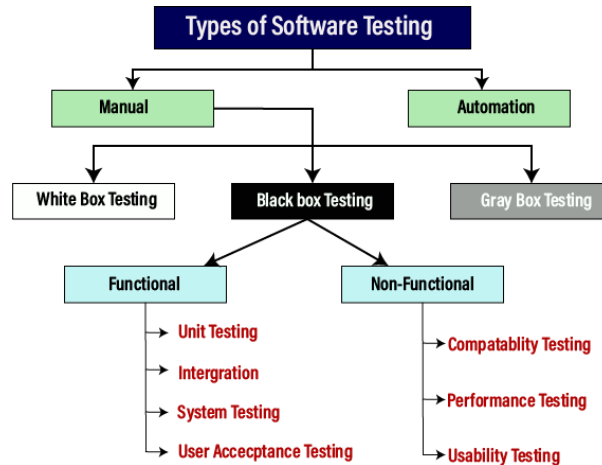
Testing

Testing is a group of techniques to determine the correctness of the application under the predefined script but, testing cannot find all the defect of application.

By testing we are ensuring quality of the software/product and as quality test engineer our objective is to provide defect free and quality software/product.

Types of Software testing

We have various types of testing available in the market, which are used to test the application or the software.



- **Manual Testing**

Testing which is performing manually on the application/software/build to check the completeness and correctness of application/software/build is called as manual testing.

Ex- Check the banking application login and password manually.

Manual testing can be further divided into three types of testing, which are as follows:

- White box testing, Black box testing and gray box testing.

- **Automation testing**

Automation testing is a process of converting any manual test cases into the test scripts with the help of automation tools, or any programming language is known as automation testing. With the help of automation testing, we can enhance the speed of our test execution because here, we do not require any human efforts. We need to write a test script and execute those scripts.

Software Quality:

Software Quality shows how good and reliable a product is. To convey an associate degree example, think about functionally correct software. It performs all functions as laid out in the [SRS document](#). But it has an associate degree virtually unusable program. even though it should be functionally correct, we tend not to think about it to be a high-quality product.

➤ **Factors of Software Quality**



1. **Portability:** A software is claimed to be transportable, if it may be simply created to figure in several package environments, in several machines, with alternative code merchandise, etc.
2. **Usability:** A software has smart usability if completely different classes of users (i.e. knowledgeable and novice users) will simply invoke the functions of the merchandise.
3. **Reusability:** A software has smart reusability if completely different modules of the merchandise will simply be reused to develop new merchandise.
4. **Correctness:** Software is correct if completely different needs as laid out in the SRS document are properly enforced.

5. **Maintainability:** A software is reparable, if errors may be simply corrected as and once they show up, new functions may be simply added to the merchandise, and therefore the functionalities of the merchandise may be simply changed, etc
6. **Reliability:** Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and also by ensuring that if failures occur, the system can handle them or can recover easily.
7. **Efficiency.** The more efficient software is, the less it uses of CPU-time, memory, disk space, [network bandwidth](#), and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU time, memory and disk usage are less of a concern than in years gone by.

Fundamentals of Test Process

Testing is a process rather than a single activity. This process starts from test planning then designing test cases, preparing for execution and evaluating status till the test closure. So, we can divide the activities within the fundamental test process into the following basic steps:

- 1) Planning and Control
- 2) Analysis and Design
- 3) Implementation and Execution
- 4) Evaluating exit criteria and Reporting
- 5) Test Closure activities

1) Planning and Control:

Test planning has following major tasks:

- 1) To determine the scope and risks and identify the objectives of testing.
- 2) To determine the test approach.
- 3) To implement the test policy and/or the test strategy. (Test strategy is an outline that describes the testing portion of the software development cycle. It is created to inform PM, testers and developers about some key issues of the testing process. This includes the testing objectives, method of testing, total time and resources required for the project and the testing environments.).
- 4) To determine the required test resources like people, test environments, PCs, etc.
- 5) To schedule test analysis and design tasks, test implementation, execution and evaluation.

2) Analysis and Design:

Test analysis and Test Design has the following major tasks:

- 1) To review the test basis. (The test basis is the information we need to start the test analysis and create our own test cases. Basically, it's a documentation on which test cases are based, such as requirements, design specifications, product risk analysis, architecture and interfaces. We can use the test basis documents to understand what the system should do once built.)
- 2) To identify test conditions.
- 3) To design the tests.
- 4) To evaluate testability of the requirements and system.
- 5) To design the test environment set-up and identify and required infrastructure and tools.

3) Implementation and Execution:

During test implementation and execution, we take the test conditions into test cases and procedures and other testware such as scripts for automation, the test environment and any other test infrastructure.

- Test implementation has the following major task:
 1. To develop and prioritize our test cases by using techniques and create **test data** for those tests.
 2. To create test suites from the test cases for efficient test execution.
 3. To implement and verify the environment.
- Test execution has the following major task:
 1. To execute test suites and individual test cases following the test procedures.
 2. To re-execute the tests that previously failed to confirm a fix. This is known as confirmation testing or re-testing.
 3. To log the outcome of the test execution and record the identities and versions of the software under tests. The test log is used for the audit trail.
 4. To Compare actual results with expected results.
 5. Where there are differences between actual and expected results, it report discrepancies as Incidents.

4) Evaluating Exit criteria and Reporting:

Based on the risk assessment of the project we will set the criteria for each test level against which we will measure the “enough testing”. These criteria vary from project to project and are known as exit criteria.

Exit criteria come into picture, when:

1. Maximum test cases are executed with certain pass percentage.
 2. Bug rate falls below certain level.
 3. When achieved the deadlines.
- **Evaluating exit criteria** has the following major tasks:
- i. To check the test logs against the exit criteria specified in test planning.
 - ii. To assess if more test are needed or if the exit criteria specified should be changed.
 - iii. To write a test summary report for stakeholders.

5) Test Closure activities:

Test closure activities are done when software is delivered. The testing can be closed for the other reasons also like:

- When all the information has been gathered which are needed for the testing.
 - When a project is cancelled.
 - When some target is achieved.
 - When a maintenance release or update is done.
- **Test closure activities** have the following major tasks:
- i. To check which planned deliverables are delivered and to ensure that all incident reports have been resolved.
 - ii. To finalize and archive testware such as scripts, test environments, etc. for later reuse.
 - iii. To handover the testware to the maintenance organization. They will give support to the software.
 - iv. To evaluate how the testing went and learn lessons for future releases and projects.

The Psychology of Testing:

In software testing, psychology plays an extremely important role. It is one of those factors that stays behind the scene, but has a great impact of the end result. Categorized into three sections, the psychology of testing enables smooth testing as well as makes the process hassle free. It is mainly dependent on the mindset of the developers and testers, as well as the quality of communication between them. Moreover, the psychology of testing improves mutual understanding among team members and helps them work towards a common goal.

The three sections of psychology of testing are:

- Mindset of Developers and Testers.
- Communication in a Constructive Manner.
- Test Independence.

➤ Mindset of Developers and Testers:

As testing and analyzing the software is much different than developing or programming it, it is quite clear that to develop a software with unique features and superior quality, developers and testers of different mindset are required. Moreover, the introduction of new development and testing techniques and methodologies has further made this a necessary requirement.

➤ Communication in a Constructive Manner:

Any process, whether technical or non-technical requires constant communication between team members to successfully achieve their goals and aims.

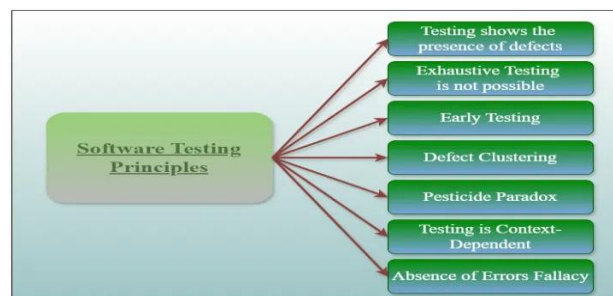
Communication, if done in a polite and courteous manner can help build a strong and reliable relationship between team members and help them avoid any misunderstanding. Similarly, during the process of testing also, the requirement of communication in constructive manner is extremely high. As testers are responsible for finding bugs and reporting them, it becomes vital for them to communicate it in a courteous, polite, and suitable way to avoid hurting and disappointing someone.

➤ Test Independence:

Test independence helps developers and other stakeholders get more accurate test results, which helps them build a better software product, with innovative and unique features and functionality.

Principles of software testing:

Software testing is an important aspect of software development, ensuring that applications function correctly and meet user expectations.



1. Testing shows the presence of defects
2. Exhaustive testing is not possible
3. Early testing
4. Defect clustering
5. Pesticide paradox
6. Testing is Context-Dependent
7. Absence of Errors fallacy

1. Testing shows the Presence of Defects

The goal of software testing is to make the software fail. Software testing reduces the presence of defects. Software testing talks about the presence of defects and doesn't talk about the absence of defects. Software testing can ensure that defects are present, but it cannot prove that software is defect-free. Even multiple tests can never ensure that software is 100% bug-free. Testing can reduce the number of defects but not remove all defects.

2. Exhaustive Testing is not Possible

It is the process of testing the functionality of the software in all possible inputs (valid or invalid) and pre-conditions is known as exhaustive testing. Exhaustive testing is impossible means the software can never test at every test case. It can test only some test cases and assume that the software is correct, and it will produce the correct output in every test case. If the software will test every test case, then it will take more cost, effort, etc., which is impractical.

3. Early Testing

To find the defect in the software, early test activity shall be started. The defect detected in the early phases of SDLC will be very less expensive. For better performance of software, software testing will start at the initial phase i.e. testing will perform at the requirement analysis phase.

4. Defect Clustering

In a project, a small number of modules can contain most of the defects. The Pareto Principle for software testing states that 80% of software defects come from 20% of modules.

5. Pesticide Paradox

Repeating the same test cases, again and again, will not find new bugs. So, it is necessary to review the test cases and add or update test cases to find new bugs.

6. Testing is Context-Dependent

The testing approach depends on the context of the software developed. Different types of software need to perform different types of testing. For example, the testing of the e-commerce site is different from the testing of the Android application.

7. Absence of Errors Fallacy

If a built software is 99% bug-free but does not follow the user requirement then it is unusable. It is not only necessary that software is 99% bug-free but it is also mandatory to fulfill all the customer requirements.

Test Metrics:

Software testing metrics are the quantifiable indexes of the testing procedure, quality, productivity, progress, and overall health. The purpose is to boost the effectiveness and conclusiveness of the software testing operation and support the fabrication of better resolutions for future testing by delivering precise data about the test proceedings.

Types of Software Testing Metrics:

Software testing metrics are split into three groups.

1. **Process Metrics:** The process metrics outline the characteristics and performance of a design. These features add to the SDLC (Software Development Life Cycle) procedure's enhancement and conservation.
2. **Product Metrics:** A product's design, size, quality, performance, and complexity are delineated by the product metrics. Developers can amend the caliber of their software development by employing these features.
3. **Project Metrics:** Project metrics are employed to evaluate the generic quality of a project. It estimates the design resources and deliverables and decides productivity, cost, and possible flaws.

Test Metrics Life Cycle

The Test Metrics Life Cycle is gathering information, looking at it, and reporting on it to determine how successful a software project is. It starts by picking the right metrics that will show progress and what needs to be fixed. Then you collect data from logs, bug-tracking systems, and performance tests. After that, you review all the information you gathered and report on it to see how well the software works. Finally, with this knowledge, changes can be made to make the product or process better. By tracking test metrics throughout a project's life cycle, companies can ensure their work is getting them closer to their goal.

Analysis:

- Recognizing the most appropriate metrics for testing.
- Defining the adopted QA standards.

Communicate:

- Training the software testing team on the data points to be collected for processing the recognized metrics.
- Informing the testing team and the stakeholders of the requirements.

Evaluation:

- Capturing and then verifying the data.
- Using the collected data for evaluating the value of the metric.

Report:

- Creating a sound and compelling inference for the paper.
- Gathering potent inputs from the stakeholders and representatives based on the information. Distribution of these reports to the representatives and stakeholders.